



AUDIT REPORT

June 2026

For



VITTAGEMS

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	11
Types of Severity	13
Types of Issues	14
Severity Matrix	15
 High Severity Issues	16
1. Discount Calculation Can Round to Zero, Allowing Tokens to Be Acquired for Free	16
 Medium Severity Issues	17
2. Stake Cap Enforced on Lifetime Count, Permanently Locking Active Users Out	17
3. Chainlink Price Feed Used Without L2 Sequencer Liveness Verification	18
4. Incomplete Chainlink Oracle Validation Allows Stale Price Acceptance	19
 Low Severity Issues	20
5. Transfer Fee Validation Applied to Reward Deposits but Not to Stake Deposit	20
6. TierPlanUpdated Event Emits Incorrect Historical Minimum Stake Value	21
7. Chainlink Circuit Breaker Can Cause Oracle to Report an Incorrect ETH Price	22
 Informational Issues	23
8. Removed Members Persist in Member List, Causing Duplicates on Re-Addition	23
9. Unsafe Token Transfer Pattern Used in Reward Distribution	24



10. Secret Word Verification Adds No Security on Top of Existing Access Control	25
Functional & Automated Tests	26
Closing Summary & Disclaimer	27



Executive Summary

Project Name	VittaGems
Protocol Type	Reserve-Backed Token / Presale / Staking / DAO Governance
Project URL	https://www.vittagems.com/
Overview	Vittagems is a reserve-backed token ecosystem built around the VGMG ERC20 token. The protocol enforces a hard invariant that total token supply can never exceed a declared reserve value, with that reserve value governed on-chain by a DAO. The ecosystem comprises four contracts: a reserve-gated ERC20 token with a 10 billion hard cap; a presale contract supporting purchases in ETH, USDT, and USDC with volume-based discount tiers; a tier-based staking contract offering fixed lock periods ranging from six to twenty-four months with corresponding reward rates; and a governance contract through which DAO members vote on reserve update proposals backed by off-chain audit documents The governance contract doubles as the reserve oracle consumed by the ERC20, making it the central trust boundary of the entire system along with onlyOwner in other part of the system.
Audit Scope	The scope of this Audit was to analyze the VittaGems Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/shailesh44/Smart-Contract-Audit-/tree/main/contracts
Branch	main
Contracts in Scope	contracts/staking.sol contracts/presale.sol contracts/governance.sol contracts/erc20.sol
Commit Hash	572120c52a45aaeb39ed9a90f95b7b2e14516fa5
Language	Solidity



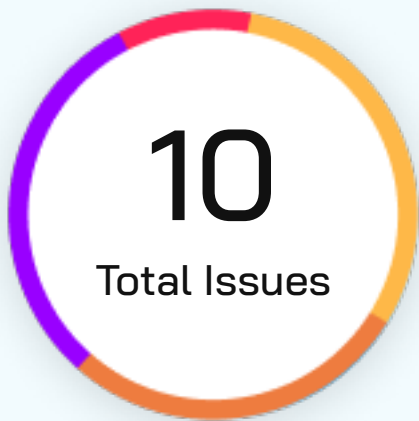
Blockchain	Base
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th May 2026 to 28th May 2026
Updated Code Received	10th June 2026
Review 2	10th June 2026
Fixed In	637dc8811598c212997ed93427f66d12b45774f8

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	1 (10%)
Medium	3 (30%)
Low	3 (30%)
Informational	3 (30%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	1	3	3	3



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Discount Calculation Can Round to Zero, Allowing Tokens to Be Acquired for Free	High	Resolved
2	Stake Cap Enforced on Lifetime Count, Permanently Locking Active Users Out	Medium	Resolved
3	Chainlink Price Feed Used Without L2 Sequencer Liveness Verification	Medium	Resolved
4	Incomplete Chainlink Oracle Validation Allows Stale Price Acceptance	Medium	Resolved
5	Transfer Fee Validation Applied to Reward Deposits but Not to Stake Deposits	Low	Resolved
6	TierPlanUpdated Event Emits Incorrect Historical Minimum Stake Value	Low	Resolved
7	Chainlink Circuit Breaker Can Cause Oracle to Report an Incorrect ETH Price	Low	Resolved
8	Removed Members Persist in Member List, Causing Duplicates on Re-Addition	Informational	Resolved



9	Unsafe Token Transfer Pattern Used in Reward Distribution	Informational	Resolved
10	Secret Word Verification Adds No Security on Top of Existing Access Controls	Informational	Resolved

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Discount Calculation Can Round to Zero, Allowing Tokens to Be Acquired for Free

Resolved

Path

presale.sol

Function Name

`getDiscountedPrice()`

Description

MIN_PRICE_CENTS is currently set to 1 (\$0.01), and because Solidity uses integer division, discounted prices at extremely low configured values can round down unexpectedly. Discount percentages are applied using integer arithmetic, which truncates results. At a base price of 1 cent, every discount tier produces a price of zero after division. A zero discounted price flows through to the total cost calculation, which also returns zero, meaning buyers can complete a purchase without sending any ETH or stablecoin. The mint call succeeds and tokens are issued at no cost. The team confirmed the intended design is absolutely not to provide free tokens.

Impact

Any buyer can acquire an unlimited number of tokens for free as long as the base price is set at or near the minimum. This results in direct financial loss to the protocol and undermines the entire token distribution model.

Likelihood

Medium. The scenario requires basePriceCents to be configured at or very close to 1. While this requires an owner action, the current contract permits it within the defined valid range, making it a realistic misconfiguration risk.

Recommendation

Either:

- Increase the minimum allowed token price to avoid invalid discount rounding scenarios, or
- Adjust the pricing math to enforce a minimum non-zero discounted price



Medium Severity Issues

Stake Cap Enforced on Lifetime Count, Permanently Locking Active Users Out

Resolved

Path

staking.sol

Function Name

`stake()`

Description

MAX_STAKES_PER_USER is enforced by checking the length of the `_userStakelds` array, which only ever grows. Unstaking marks a position as completed but never removes its ID from the array. A user who has staked and fully exited 100 positions over time will have an array of length 100 and zero active stakes, yet every future attempt to stake reverts permanently with `MaxStakesReached()`. There is no admin override or recovery path. The contract already maintains an `activeStakeCount` field in `UserAggregates` that correctly tracks concurrent open positions.

Impact

Long-term protocol participants who cycle through staking positions can be permanently locked out of staking despite having no active positions.

Likelihood

Medium. For casual users this limit may never be reached. For active participants or automated strategies cycling through positions, it is a realistic outcome with extended protocol use.

Recommendation

Change the stake guard to check `_userAggregates[msg.sender].activeStakeCount` instead of `_userStakelds[msg.sender].length`. No structural changes are required, this counter is already maintained correctly throughout the contract.



Medium Severity Issues

Chainlink Price Feed Used Without L2 Sequencer Liveness Verification

Resolved

Path

presale.sol

Function Name

`getNativeAmountFromUsd()`

Description

The contract is to be deployed on Base, an L2 network where transaction ordering depends on a centralized sequencer. When the sequencer goes offline, Chainlink price feeds on-chain stop updating. When the sequencer recovers, there is a window where transaction processing resumes before the price feed catches up to current market conditions. An attacker monitoring the recovery can submit purchases within this window at a price that no longer reflects live market rates. Chainlink provides a dedicated Sequencer Uptime Feed for Base to detect this condition.

Impact

An attacker can exploit the post-recovery window to purchase tokens at a stale, favorable ETH price, underpaying relative to the token's intended USD value.

Likelihood

Low. Sequencer outages on Base are infrequent but have occurred. The attack requires precise timing and monitoring, limiting it to sophisticated actors.

Recommendation

Integrate the Chainlink Sequencer Uptime Feed into `getNativeAmountFromUsd()`.



Medium Severity Issues

Incomplete Chainlink Oracle Validation Allows Stale Price Acceptance

Resolved

Path

presale.sol

Function Name

`getNativeAmountFromUsd()`

Description

The `latestRoundData()` call returns five values, but only the price and timestamp are validated. The `roundId` and `answeredInRound` identifiers are discarded. When a Chainlink round fails to reach consensus, the oracle carries the previous round's answer forward, `answeredInRound` becomes less than `roundId` in this state. The timestamp check alone does not detect this; the price can appear fresh while actually belonging to a prior round. The contract would accept and price ETH purchases against stale data without any indication of the problem.

Impact

Buyers could purchase tokens at a mispriced ETH rate during oracle degradation events, resulting in underpayment and direct loss to the protocol.

Likelihood

Low. Round incompleteness requires oracle node failures or sustained network congestion. The window of exposure is typically short, but it represents a known and documented Chainlink integration gap.

Recommendation

Destructure all five return values from `latestRoundData()` and add a check that `answeredInRound >= roundId`.



Low Severity Issues

Transfer Fee Validation Applied to Reward Deposits but Not to Stake Deposits

Resolved

Path

staking.sol

Function Name

`stake()`

Description

`depositRewardPool()` uses a before/after balance comparison to detect fee-on-transfer tokens, reverting if the received amount differs from what was requested. `stake()`, which pulls the same staking token inward, performs no equivalent check and immediately credits the full requested amount to the user's position and the global total. If the staking token carries a transfer fee, every stake would overstate the contract's actual holdings. The accounting shortfall would compound silently and eventually cause withdrawals to fail. Since `stakingToken` is immutable, the risk is bounded to the token configured at deployment, but the inconsistency within the same contract is a design quality gap.

Impact

If the staking token has transfer fees, user balances would be overcredited relative to actual holdings, leading to withdrawal failures for later users in a first-in, first-out manner.

Likelihood

Low. The risk is only realized if the configured staking token has transfer fees, which requires a deliberate deployment decision or a token upgrade outside the contract's control.

Recommendation

Apply the same balance-difference validation in `stake()` as already exists in `depositRewardPool()`, and credit user accounting with the actual received amount rather than the requested amount.



Low Severity Issues

TierPlanUpdated Event Emits Incorrect Historical Minimum Stake Value

Resolved

Path

staking.sol

Function Name

`_updateSingleTierPlan()`

Description

The TierPlanUpdated event includes an oldMinStake parameter intended to record the previous minimum stake. The intended behavior was to emit the actual old value before updating the tier plan so off-chain indexers and analytics systems can reconstruct configuration history accurately. However, in `_updateSingleTierPlan()`, the new value is passed for both the old and new minimum stake parameters. The previous value is never captured and is lost permanently at the point of each update. Any off-chain system, indexer, or dashboard relying on this event to track the history of minimum stake changes will receive incorrect data.

Impact

Any off-chain system, indexer, or dashboard relying on this event to track the history of minimum stake changes will receive incorrect data.

Likelihood

High. This happens unconditionally every time `_updateSingleTierPlan()` is called with a new minimum stake value, there is no code path that avoids the incorrect emission.

Recommendation

Capture the old minimum stake into a local variable before modifying storage, then pass that variable as the oldMinStake argument in the event.



Low Severity Issues

Chainlink Circuit Breaker Can Cause Oracle to Report an Incorrect ETH Price

Resolved

Path

presale.sol

Function Name

`getNativeAmountFromUsd()`

Description

Chainlink aggregators are configured with hardcoded `minAnswer` and `maxAnswer` bounds. If the real market price moves outside these bounds, the oracle continues returning the boundary value rather than the actual price. From the contract's perspective, the returned value passes all checks, it is positive, fresh, and within the staleness window. The contract cannot distinguish a circuit breaker value from a real observation. During a significant ETH price crash, the oracle would report a price above market, causing the ETH cost calculation to undercharge buyers.

Impact

During extreme market events, buyers pay less ETH than the token's intended USD price demands. The difference represents a direct subsidy borne by the protocol.

Likelihood

Low. Circuit breaker conditions require severe and rapid price movement. These events are rare but have materialized in real market conditions.

Recommendation

Implement a secondary price sanity check such as a deviation threshold against a TWAP, or an explicit comparison against the aggregator's `minAnswer` and `maxAnswer` values. At minimum, ensure the owner can pause ETH-based purchases independently during extreme market conditions, preserving stablecoin purchases as a fallback.



Removed Members Persist in Member List, Causing Duplicates on Re-Addition

Resolved

Path

governance.sol

Function Name

`removeMember()`

Description

memberList is an append-only array. When a member is removed, their isActive flag is set to false and the active count is decremented, but the address remains in the array indefinitely. If the same address is re-added later, it is pushed onto memberList a second time. The addMember() guard only checks isActive, which passes for a previously removed member, so re-addition is permitted without restriction. Any external system consuming getMemberList() would encounter duplicate addresses and would be unable to derive an accurate current membership set from the array alone.

Recommendation

Consider preventing duplicate pushes when a removed member is re-added



Unsafe Token Transfer Pattern Used in Reward Distribution

Resolved

Path

VotingEscrow.sol, RewardDistributor.sol

Description

Both `claimReward()` and `withdrawVGMG()` use `raw transfer()` wrapped in `require()` to disburse VGMG tokens. This pattern assumes the token returns a boolean on transfer. While the ERC20 standard mandates this, several widely deployed tokens – most notably USDT – do not return a value, causing the `require()` call to revert incorrectly. VGMG is a standard OpenZeppelin ERC20 that does return `bool`, so this is not exploitable in the current deployment. However, it is a deviation from accepted best practice that would become a live failure if the token reference were pointed at a non-compliant token before the address lock.

Recommendation

Replace both `transfer()` calls with `safeTransfer()` from OpenZeppelin's SafeERC20 library, which correctly handles tokens that do not return a value.



Secret Word Verification Adds No Security on Top of Existing Access Controls

Resolved

Path

governance.sol

Function Name

`createProposal()`

Description

`createProposal()` takes a plaintext `secretWord` in calldata, hashes it, and stores the hash on the proposal. `executeReserveUpdate()` requires the caller to supply the matching plaintext to proceed. This is intended as an additional execution gate, but it provides no real protection. The secret is submitted as public calldata, readable by anyone who observes the `createProposal()` transaction, it is never actually secret. Furthermore, `executeReserveUpdate()` is already restricted to `DEFAULT_ADMIN_ROLE` or `RESERVE_UPDATE_ROLE` by an access control check that runs before the secret verification. The role check is the genuine security boundary; the secret check adds no restriction on top of it. The team confirmed the mechanism is a residual from an earlier design iteration and is no longer aligned with the current access control model.

Recommendation

Team comment the existing role-based access control is the correct and sufficient authorization layer and the `secretWord` can likely be removed entirely without materially affecting the security model, simplifying proposal execution and reducing operational complexity.



Functional Tests

Some of the tests performed are mentioned below:

- ✓ `buyTokensWithNative()` mints the correct token amount and refunds excess ETH to the buyer
- ✓ `stake()` creates a position with the correct tier parameters, lock duration, and reward rate snapshot
- ✓ `claimReward()` advances the cycle cursor and transfers the correct reward amount per completed cycle
- ✓ Access control role-restricted functions reject calls from unauthorized addresses across all contracts
- ✓ `emergencyWithdraw()` returns principal only and is rejected when the contract is not paused
- ✓ `lockReserveOracle()` permanently prevents further oracle address changes after being called
- ✓ `pause()` and `unpause()` correctly block and restore token transfers, purchases, staking operations, and proposal activity

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of VittaGems. We performed our audit according to the procedure described above.

1 High , 3 Medium , 3 Low, 3 Informational issues were found, which have been noted by the team. In the end,Vittagems team resolved all issues.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



VITTAGEMS



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com